

Orchestrator Platform

Backend Architecture & Working Flow

Updated: 2026-08-18

Repository: Accuknox / Orchestrator

Multi-tenant SaaS orchestrator: FastAPI API, ARQ worker, PostgreSQL, Redis, Qdrant RAG, OpenAI-compatible LLM gateway, agents, workflows, MCP HTTP, knowledge bases, API keys, runs/trace, and webhooks.

1. Overview

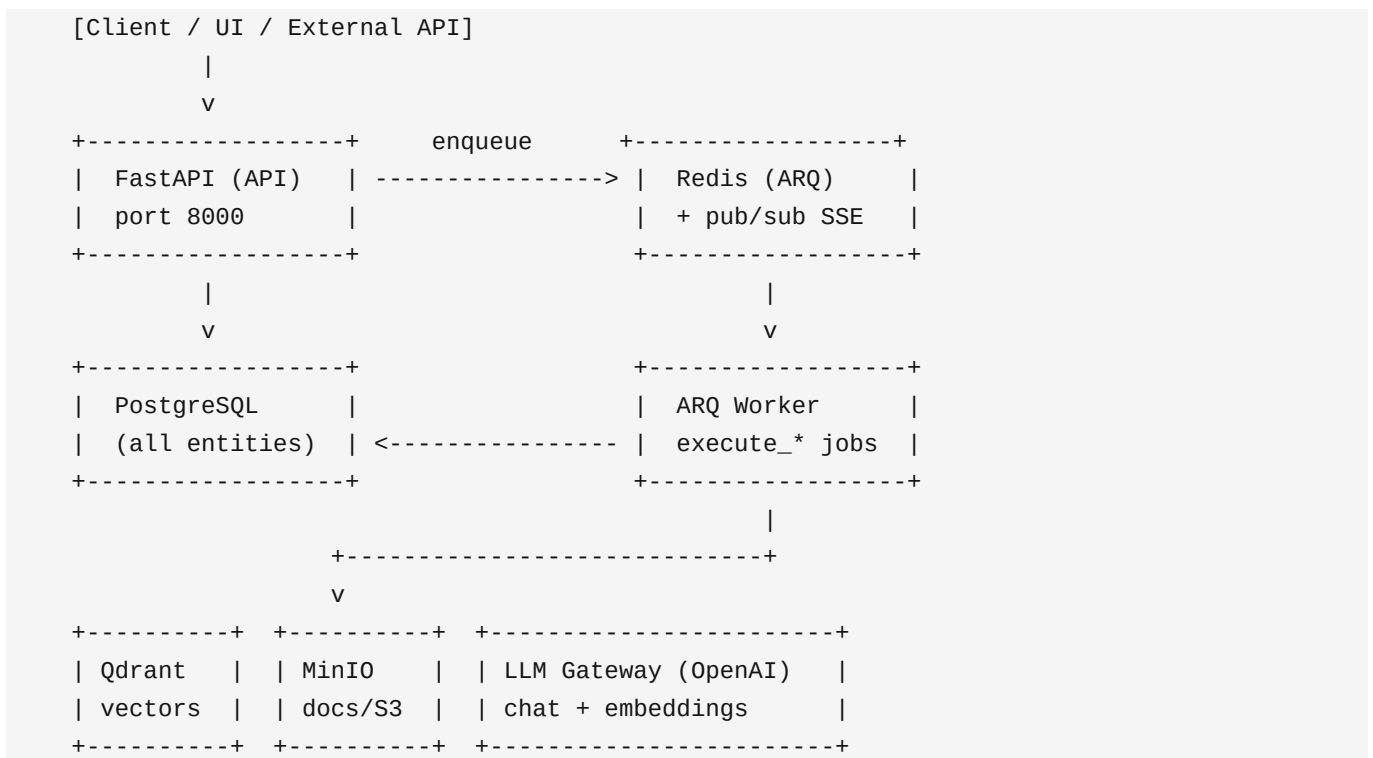
Orchestrator is a multi-agent orchestration platform designed as multi-tenant SaaS. The backend exposes a REST API (FastAPI) for configuration and invocation. Long-running work (agent runs, workflow runs, document indexing) is delegated to an ARQ worker via Redis. All tenant data is scoped by `organization_id`.

Core capabilities

- JWT login and scoped API keys (oak_...) for external invoke
- Agents with system prompts, models, RAG knowledge bases, and MCP tools
- Workflow graphs: agent, supervisor, tool, branch, parallel, human-in-the-loop nodes
- Async invoke returns 202 + `run_id`; poll GET `/runs/{id}` or SSE `/runs/{id}/events`
- Per-org LLM gateway config (OpenAI-compatible) with SSL/proxy controls
- Usage tracking, webhooks on run completion, RBAC roles

2. System Architecture

Three runtime processes plus infrastructure services. The API is stateless; the worker owns execution. Redis carries job queue and real-time run events.



Technology stack

Layer	Technology
API	FastAPI, Uvicorn, Pydantic v2
Worker	ARQ (async Redis queue)
Database	PostgreSQL 16, SQLAlchemy 2 async, Alembic
Queue / Events	Redis 7 (jobs + pub/sub for SSE)
Vectors	Qdrant (per-org collections)
LLM	OpenAI-compatible HTTP (Ollama, Bifrost, corporate gateway)
Auth	JWT (users) + hashed API keys

3. Project Structure

```

Accuknox/
  apps/
    api/          orchestrator_api - FastAPI routes, deps, guards
    worker/       orchestrator_worker - ARQ task functions
    web/          React frontend (not covered here)
  packages/
    core/         models, config, DB, security, RBAC, usage
    llm/          OpenAI client, gateway resolution, embeddings
    rag/          chunking, Qdrant store
    mcp/          MCP HTTP client, tool discovery
    runtime/      agent + workflow execution engine
    events/       Redis EventPublisher for run SSE
  infra/         docker-compose (postgres, redis, qdrant, minio)
  scripts/       migrate.sh, cloud-agent scripts
  tests/         integration tests (pytest)

```

4. Authentication & Authorization

Auth modes

- Bearer JWT: from POST /v1/auth/login or /signup; contains sub, org_id, role
- X-API-Key or Bearer oak_...: scoped API key with scopes and optional resource_ids

AuthContext (deps.py) resolves org_id for every request. API keys cannot access settings, knowledge upload, or MCP management unless scope allows invoke/read.

RBAC roles (rbac.py)

Role	Summary
owner	Full org control including org:write, api keys, all resources
admin	Like owner minus org:write
builder	Create/edit agents, workflows, KB, MCP; invoke runs
viewer	Read-only across agents, workflows, KB, MCP, runs

API key scopes

- agent:invoke, workflow:invoke, run:read (typical external integration)
- resource_ids limits key to specific agent/workflow UUIDs
- rate_limit_per_minute per key; org-level limit via ORG_RATE_LIMIT_PER_MINUTE

5. API Routes Summary

All business routes are under /v1. Health check: GET /health

Prefix / Route	Description
POST /v1/auth/signup	Create org + owner user, return JWT
POST /v1/auth/login	Email/password login
GET /v1/auth/me	Current user profile
CRUD /v1/agents	Agent management; POST /{id}/publish
POST /v1/agents/{id}/invoke	Async agent run (202, run_id)
CRUD /v1/workflows	Graph workflows; validate + publish
POST /v1/workflows/{id}/invoke	Async workflow run
POST /v1/runs/{id}/resume	Resume human-in-the-loop step
GET /v1/runs, /runs/{id}	List and poll run status
GET /v1/runs/{id}/events	SSE stream of run events
CRUD /v1/knowledge-bases	KB + document upload/index
CRUD /v1/mcp-connections	MCP HTTP servers + discover tools
CRUD /v1/api-keys	Create/list/revoke API keys
GET /v1/usage	Daily usage aggregates
GET/PUT/DELETE /v1/settings/llm-gateway	Per-org gateway config
POST /v1/settings/llm-gateway/test	Ping gateway via chat completion

6. Agent Invoke Flow

End-to-end path when a client calls POST /v1/agents/{agent_id}/invoke:

1. API: authenticate, rate limit, check agent exists + published (if API key)
2. API: create Run row (status=pending, input={message, context})
3. API: enqueue ARQ job execute_agent_run(run_id) on Redis
4. API: return 202 { run_id, status: pending }
5. Worker: load Run, set status=running, publish run.status event
6. Worker: load Agent, get_gateway_for_org(), build_agent_config()
7. Worker: load_mcp_clients() for org
8. Worker: execute_agent() - RAG retrieval, MCP tool loop, LLM chat
9. Worker: on each token/event -> Redis pub/sub + append run.trace
10. Worker: set status=completed/failed, record usage, webhook POST
11. Client: poll GET /runs/{id} or SSE GET /runs/{id}/events

execute_agent (runtime/agent.py)

- Embeds user query; searches Qdrant for bound knowledge_base_ids
- Injects retrieved chunks into system context
- If MCP tools bound: OpenAI function-calling loop (max 8 rounds)
- Returns output text + metrics (tokens_in, tokens_out)

7. Workflow Execution Flow

Workflows store a JSON graph: entry node id, nodes array, edges array. Invoke enqueues `execute_workflow_run`. Node types supported in runtime:

Node type	Behavior
agent	Run linked agent or inline prompt/model config
supervisor	LLM routes to child agents (children list)
tool	Call MCP tool on <code>connection_id</code> + <code>tool_name</code>
branch	Conditional edges (<code>route == value, variables.key</code>)
parallel	Run children concurrently (<code>asyncio.gather</code>)
human	Pause run; <code>status=awaiting_input</code> ; checkpoint saved

Human-in-the-loop resume

When a human node is hit, `run.status` becomes `awaiting_input` and `checkpoint_data` stores `WorkflowState`. Client calls `POST /v1/runs/{id}/resume` with `human_response`. Worker job `resume_workflow_run` restores state, appends human input, continues from next node.

8. Knowledge Base & RAG Flow

Document upload path:

1. `POST /v1/knowledge-bases/{kb_id}/documents` (multipart file)
2. File saved to storage path (`org_id/kb_id/filename`; dev uses local `/tmp`)
3. Document row created (`status=pending`)
4. ARQ job `index_document(document_id)` enqueued
5. Worker: read file, `chunk_text()`, embed via gateway `embed_model`
6. Worker: `QdrantStore.upsert_chunks(org, kb, doc, vectors)`
7. Document status -> indexed (or failed)

At agent runtime, `execute_agent` queries Qdrant with the user message embedding and injects top chunks into the LLM context for agents with `knowledge_base_ids` in config.

9. MCP HTTP Connections

- CRUD `/v1/mcp-connections` stores `base_url`, `auth` (encrypted), `tool_allowlist`
- `POST /{id}/discover` fetches tools from remote MCP HTTP server
- `POST /{id}/test` verifies connectivity
- Agents bind tools via `config.mcp_tools: [{connection_id, tools: [...]}]`
- Worker loads clients per org; runtime maps tools to OpenAI function schemas

10. LLM Gateway Configuration

Platform default from env: LLM_GATEWAY_URL, LLM_GATEWAY_KEY, LLM_DEFAULT_MODEL, EMBED_MODEL. Per-org override stored in llm_gateway_configs table (one row per org).

Key modules

- orchestrator_llm.client: normalize_gateway_base_url(), create_openai_client()
- httpx verify=LLM_GATEWAY_VERIFY_SSL, trust_env=LLM_GATEWAY_TRUST_ENV
- orchestrator_llm.gateway: get_gateway_for_org(), upsert_gateway_config()

Corporate / self-signed gateway (.env)

```
LLM_GATEWAY_URL=https://aigw.example.net/v1
LLM_GATEWAY_VERIFY_SSL=false      # like curl -k
LLM_GATEWAY_TRUST_ENV=false       # bypass HTTP_PROXY on Windows
```

Settings UI: PUT /v1/settings/llm-gateway saves normalized base URL. POST /v1/settings/llm-gateway/test sends a ping chat completion and returns ok/error + hint.

11. ARQ Worker Tasks

Job function	Trigger	Purpose
execute_agent_run	Agent invoke	Run single agent LLM + RAG + MCP
execute_workflow_run	Workflow invoke	Execute graph from entry node
resume_workflow_run	Run resume	Continue after human node
index_document	Document upload	Chunk, embed, upsert to Qdrant

Start worker: arq orchestrator_worker.tasks.WorkerSettings

12. Real-time Events & Webhooks

SSE (Server-Sent Events)

EventPublisher publishes to Redis channel run:{run_id}:events. GET /runs/{id}/events subscribes and streams JSON: run.status, run.completed, run.failed, run.awaiting_input, token events during execution.

Webhooks

Invoke body may include webhook_url and webhook_secret. On completion/failure/awaiting_input, worker POSTs signed payload (X-Orchestrator-Signature) to the callback URL.

13. Database Entities (PostgreSQL)

Table	Purpose
organizations	Tenant root (name, slug)
users	Email login, role, org FK
agents	Agent config, model, system_prompt, JSONB config
workflows	Graph JSONB, publish version

runs

Migrations: Alembic in packages/core/alembic. Run: ./scripts/migrate.sh

14. Environment Variables

Variable	Default / notes
DATABASE_URL	postgresql+asyncpg://...localhost:5432/orchestrator
REDIS_URL	redis://localhost:6379/0
LLM_GATEWAY_URL	OpenAI-compatible base URL (/v1)
LLM_GATEWAY_KEY	API key for gateway
LLM_DEFAULT_MODEL	Platform default chat model
EMBED_MODEL	nomic-embed-text typical
QDRANT_URL	http://localhost:6333
S3_*	MinIO endpoint and bucket
JWT_SECRET	Change in production
ENCRYPTION_KEY	Secrets encryption (API keys, MCP, webhooks)
LLM_GATEWAY_VERIFY_SSL	true; set false for self-signed
LLM_GATEWAY_TRUST_ENV	false; ignores system proxy
ORG_RATE_LIMIT_PER_MINUTE	120
CORS_ORIGINS	http://localhost:5173

15. Local Development

```
cd infra && docker compose up -d          # postgres, redis, qdrant, minio
cp .env.example .env && edit secrets
uv sync --all-packages && ./scripts/migrate.sh
# Terminal 1: API
uv run --package orchestrator-api uvicorn orchestrator_api.main:app --reload --port 8000
# Terminal 2: Worker
uv run --package orchestrator-worker arq orchestrator_worker.tasks.WorkerSettings
```

Production Docker

infra/docker-compose.prod.yml builds API, worker, web images. Web proxies /v1 to API.

16. Run Status Lifecycle

Status	Meaning
pending	Created; job queued
running	Worker executing
completed	Success; output available
failed	Error in run.error
awaiting_input	Workflow paused at human node
cancelled	Reserved for future use

17. External Invoke Example

```
curl -X POST http://localhost:8000/v1/agents/{agent_id}/invoke \
  -H 'X-API-Key: oak_...' \
  -H 'Content-Type: application/json' \
```



```
-d '{"input": "Hello", "context": {}}'  
# Response: 202 {"run_id": "...", "status": "pending"}  
curl http://localhost:8000/v1/runs/{run_id} -H 'X-API-Key: oak_...'
```